TI202 - Algorithmique et Structures de données 1

TD2 - Structures de contrôle (2h)

Consignes:
- Le travail est **individuel**.
- Le travail est à faire sur papier.
- Il est fortement recommandé de se référer aux cours et aux supports de cours pour répondre aux questions.

Acquis d'apprentissage:
À la fin de ce TD, vous serez capable de:
- Représenter la trace d'exécution d'un programme comportant des structures de contrôle.
- Maîtriser la syntaxe des structures de contrôle en C.
- Faire la différence entre les différents types de boucles en C.
- Utiliser les boucles pour résoudre des problèmes simples et choisir la boucle la plus appropriée pour un problème donné.

Trace d'exécution d'un programme comportant une boucle `for`

La boucle `for` est une structure de contrôle qui permet de répéter un bloc d'instructions un nombre fixe de fois. Elle est composée de trois parties: l'initialisation, la condition d'arrêt et l'incrémentation (ou décrémentation). La syntaxe de la boucle `for` est la suivante:

```
for (initialisation; condition; incrémentation) {
    // Bloc d'instructions
}
```

On rappelle que l'incrémentation est toujours exécuté à la fin de l'exécution du bloc d'instructions de la boucle. L'exemple suivant montre comment vous devez représenter la trace d'exécution d'un programme comportant une boucle `for`:

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5      for (; i < 3; i++) {
6          printf("%d ", i);
7      }
8      return 0;
9  }
```

Notez que l'incrémentation s'effectue à la ligne 7.

Lignes	i	Sortie
4	0	
6		0
7	1	
6		0 1
7	2	
6		0 1 2
7	3	
8		**Fin**

Échauffement

Exercice 1. Représenter la trace d'exécution du programme suivant en supposant que les entrées lues sont dans l'ordre:

```
5
1 2 3 4 5
```

```c
#include <stdio.h>

int main() {
    int n = 0;
    scanf("%d", &n);
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        printf("%d ", val);
    }
    return 0;
}
```

Exercice 2. Représenter la trace d'exécution du programme suivant et déduire son rôle. On suppose que la valeur lue pour n est 4.

```c
#include <stdio.h>

int main() {
    int n = 0, x = 0;
    float y = 0.0;
    scanf("%d", &n);

    for (int i = 1; i <= n; i++) {
        x += i;
    }
    y = (float) x / n;
    printf("%f\n", y);
    return 0;
}
```

Exercice 3. Représenter la trace d'exécution du programme suivant. On suppose que les valeurs suivantes seront (*éventuellement*) lues dans l'ordre:

```
1 2 3 4 5 6 7 8 9 10
```

```c
#include <stdio.h>

int main() {
    int val = 0;
    do {
        scanf("%d", &val);
    } while (val % 3 != 0 || val <= 3);
    printf("%d\n", val);
    return 0;
}
```

Exercice 4. Pour chacun des programmes suivants, déterminer la valeur de la variable i à la fin de l'exécution du programme. On suppose que la valeur lue pour a et b sont respectivement 2 et 10.

(a)
```c
#include <stdio.h>

int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    int i = 0;
    for (; a < b; a += 2) {
        i++;
    }
    return 0;
}
```

(b)
```c
#include <stdio.h>

int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    int i = 0;
    for (; a < b;) {
        i++;
        a += 2;
    }
    return 0;
}
```

(c)
```c
#include <stdio.h>

int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    int i = 0;
    while (a < b) {
        a += 2;
        i++;
    }
    return 0;
}
```

(d)
```c
#include<stdio.h>

int main() {
    int a, b;
    scanf("%d %d", &a, &b);
    int i = 0;
    do {
        a += 2;
        i++;
    } while (a < b);
    return 0;
}
```

Renforcement

Dans tous les exercices suivants, les entrées seront lue à l'aide de la fonction `scanf` et les sorties seront affichées à l'aide de la fonction `printf`.

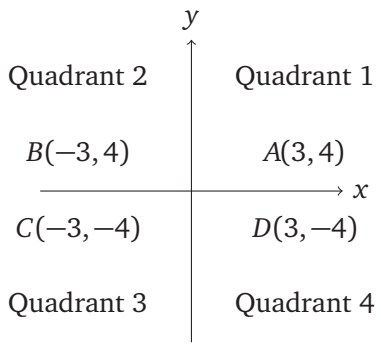
Exercice 5. Alice et Bob jouent à un jeu de dés. Chaque joueur lance un dé à six faces et le joueur qui obtient le plus grand nombre gagne. Si les deux joueurs obtiennent le même nombre, le jeu est déclaré nul. Écrire un programme qui prend en entrée les deux nombres obtenus par Alice et Bob et affiche le nom du gagnant; si le jeu est nul, le programme affichera la chaîne "Draw". Par exemple, si Alice obtient 5 et Bob obtient 3, le programme doit afficher `"Alice"`. Si Alice obtient 4 et Bob obtient 4, le programme doit afficher `"Draw"`.

Exercice 6. Écrire un programme qui prend en entrée deux entiers a et b et affiche la valeur absolue de leur différence. Par exemple, si $a = 5$ et $b = 7$, le programme doit afficher `2`; et si $a = 7$ et $b = 5$, le programme doit également afficher `2`.

Exercice 7. Alice et Bob décident de jouer à un nouveau jeu de cailloux. n cailloux sont placés sur le sol formant une séquence. Les cailloux sont numérotés de 1 à n. Tour à tour, Alice et Bob doivent prendre exactement deux cailloux de numéros consécutifs jusqu'à ce qu'il n'y ait plus de cailloux consécutifs à prendre (soit il n'y a plus qu'un seul cailloux, soit il n'y en a plus du tout). Dans ce cas, à chaque tour chaque joueur doit prendre le caillou i et $i+1$ où $1 \leq i \leq n-1$. Si le nombre de caillou restant est impair, Alice gagne; sinon c'est Bob qui gagne.

En supposant qu'Alice commence et que les deux joueurs jouent de manière optimale, écrire un programme qui prend en entrée le nombre de cailloux n et affiche le nom du gagnant. Par exemple, si $n = 1$, le programme doit afficher `"Alice"`; si $n = 2$, le programme doit afficher `"Bob"` et si $n = 5$, le programme doit afficher `"Alice"`.

Exercice 8. Il y a quatre quadrants dans le plan cartésien, numérotés de 1 à 4 dans le sens anti-horaire, tel que montré ci-dessous:



Par exemple, un point A de coordonnées $(3, 4)$ appartient au quadrant 1; un point B de coordonnées $(-3, 4)$ appartient au quadrant 2; un point C de coordonnées $(-3, -4)$ appartient au quadrant 3; et un point D de coordonnées $(3, -4)$ appartient au quadrant 4. Écrire un programme qui prend en entrées les coordonnées x et y d'un point et affiche le numéro du quadrant auquel il appartient. Par exemple, si les coordonnées du point sont $(3, 4)$, le programme doit afficher `1`.

Exercice 9. En maternelle, Alice s'amusait souvent à demander l'âge de ses camarades de classe. Après leur avoir tous demandé, elle lors donnait le nombre d'année d'écart entre le plus jeune et le plus âgé. Écrire un programme qui prend en entrée le nombre n d'élèves dans la classe, puis les âges de chaque élève et affiche la différence d'âge entre le plus jeune et le plus vieux. Par exemple, si $n = 5$ et les âges sont 5, 3, 7, 2 et 4, le programme doit afficher `5`.

Indication : *En maternelle, il n'y avait pas d'élève de plus de 99 ans :); et généralement, à 0 ans, on n'est pas encore à l'école.*

Exercice 10. Alice et Bob ont reçus respectivement a et b bonbons de la part de leur oncle Charlie. Comme Alice est la plus âgée elle en a reçu plus que Bob. Ils décident toutefois de partager équitablement les bonbons qu'ils ont reçu: Alice donnera un bonbon à Bob tant que le nombre de bonbons qu'elle a reçu est supérieur à celui de Bob. Écrire un programme qui prend en entrées le nombre de bonbons reçus par Alice et le nombre de bonbons reçus par Bob et affiche le nombre de bonbons qu'ils ont chacun après le partage. Par exemple, si Alice a reçu 5 bonbons et Bob a reçu 3 bonbons, le programme doit afficher `4 4`.

Exercice 11. Dans sans ancien studio, Alice avait uniquement une prise murale pour brancher ses appareils électriques. Afin de faire fonctionner tous ses appareils, elle a dû acheter plusieurs multiprises en les branchant entre-elles. Écrire un programme qui prend en entrée, dans un premier temps le nombre n de multiprises qu'Alice a à sa disposition, puis dans un second temps les n nombres de prises de chaque multiprises. Le programme doit ensuite afficher le nombre total d'appareils (autres qu'une multiprise) qu'Alice peut brancher simultanément. Notez que si Alice n'a aucune multiprise, elle ne pourra brancher qu'un seul appareil (sur la prise murale) et qu'une prise multiple doit être alimentée avant d'alimenter à son tour d'autres appareil.

Ci-dessous des exemples d'entrées/sortie:

Entrées 1

```
3
2 3 4
```

Sortie 1

```
7
```

Entrées 2

```
5
2 2 2 2 2
```

Sortie 2

```
6
```

Approfondissement

Exercice 12. Alice possède une petite collection de n soldats de plomb. Elle a une manière particulière de les aligner sur son étagère : elle les place en lignes de telle sorte que la ligne i contienne i soldats. On notera qu'il se peut qu'après avoir placé k soldats, les $n-k$ soldats restants ne soient plus suffisants pour former une ligne complète (selon sa méthode), dans ce cas-là, elle les range et ne les dispose pas sur l'étagère.

Écrire un programme qui prend en entrée le nombre de soldats n et affiche le nombre de lignes sur lesquels les soldats d'Alice sont alignés. Par exemple, si $n = 3$, le programme doit afficher `2`. Si $n = 6$, le programme doit afficher `3` (mais aussi pour $n = 7$, $n = 8$, $n = 9$).

Exercice 13. Alice possède une imprimante 3D qui peut tout imprimer. Elle voudrait produire n figurines pour son prochain anniversaire (en cadeau pour ses invités). Imprimer les figurines une par une peut prendre beaucoup de temps, elle trouve donc une astuce pour aller plus vite: imprimer une imprimante 3D afin de produire des figurines ou éventuellement d'autres imprimantes. Chaque impression dure un jour (peu importe ce qui est imprimé). Si elle décide d'imprimer une imprimante, celle-ci ne pourra être utilisée que le lendemain.

Écrire un programme qui prend en entrée le nombre n de figurines à imprimer et affiche le nombre minimum de jours nécessaires pour imprimer au moins n figurines. Par exemple, si $n = 1$, le programme doit afficher `1`; si $n = 5$, le programme doit afficher `4`.

Indication : *La stratégie optimale est de toujours imprimer une imprimante afin d'en avoir au moins `n` puis d'imprimer les `n` figurines en une seule journée.*

Exercice 14. Alice aime bien s'amuser avec les nombres. Voici une petite liste des choses qu'elle aime en faire. Aidez-la à les réaliser en écrivant les programmes correspondants:

(a) Une de ses activités préférées est de compter le nombre de chiffres d'un entier aussi grand soit il. Écrire un programme qui lit un entier $n >= 0$ et affiche le nombre de chiffres de n. Par exemple, si $n = 12345$, le programme doit afficher `5`; si $n = 0$, le programme doit afficher `1`.

(b) Une autre chose qu'elle aime faire est de calculer la somme des chiffres d'un entier. Écrire un programme qui lit un entier $n >= 0$ et affiche la somme des chiffres de n. Par exemple, si $n = 12345$, le programme doit afficher `15`; si $n = 0$, le programme doit afficher `0`.

(c) Alice aime bien également citer les chiffres d'un entier dans l'ordre inverse. Écrire un programme qui lit un entier $n >= 0$ et affiche les chiffres de n dans l'ordre inverse. Par exemple, si $n = 12345$, le programme doit afficher `"54321"`; si $n = 0$, le programme doit afficher `0`.

Indication : *Ici, on ne vous demande pas de calculer le nombre inversé mais uniquement d'afficher les chiffres qui le composent dans l'ordre inverse.*

(d) Comme Alice est pleine de ressource, elle est capable de calculer le nombre inversé d'un entier. Écrire un programme qui lit un entier $n >= 0$ et affiche le nombre inversé de n. Par exemple, si $n = 12345$, le programme doit afficher `54321`; si $n = 0$, le programme doit afficher `0`.

Indication : *Dans cette question, vous devez calculer le nombre inversé de `n`.*